Practical – 14

Aim – 1 : Assemble and test

Build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.

Code :

Main.java

```java
import java.util.ArrayList;

public class StudentService {

    private ArrayList<Student> students = new ArrayList<>();

    public void addStudent(Student student) {
        if (student == null) {
            throw new IllegalArgumentException("Student cannot be null");
        }

        students.add(student);
    }

    public int getCount() {
        return students.size();
    }

    public void removeStudent(int index) {
        if (index < 0 || index >= students.size()) {
            throw new IllegalArgumentException("Invalid student index");
        }

        students.remove(index);
    }
}
```

Main.java

```
}

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        StudentService service = new StudentService();

        while (true) {

            System.out.println("\n===== Student Management =====");
            System.out.println("1. Add Student");
            System.out.println("2. Show Student Count");
            System.out.println("3. Remove Student");
            System.out.println("4. Exit");

            System.out.print("Enter your choice: ");
            int choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {

                case 1:
                    System.out.print("Enter student name: ");
                    String name = sc.nextLine();

                    service.addStudent(new Student(name));

                    System.out.println("Student added successfully.");
```

ady >_ Terminal

Main.java

```
public class Main {
    public static void main(String[] args) {
        while (true) {
            switch (choice) {
                case 1:
                case 2:
                    System.out.println("Student Count: "
                            + service.getStudentCount());
                    break;

                case 3:
                    System.out.print("Enter student index: ");
                    int index = sc.nextInt();

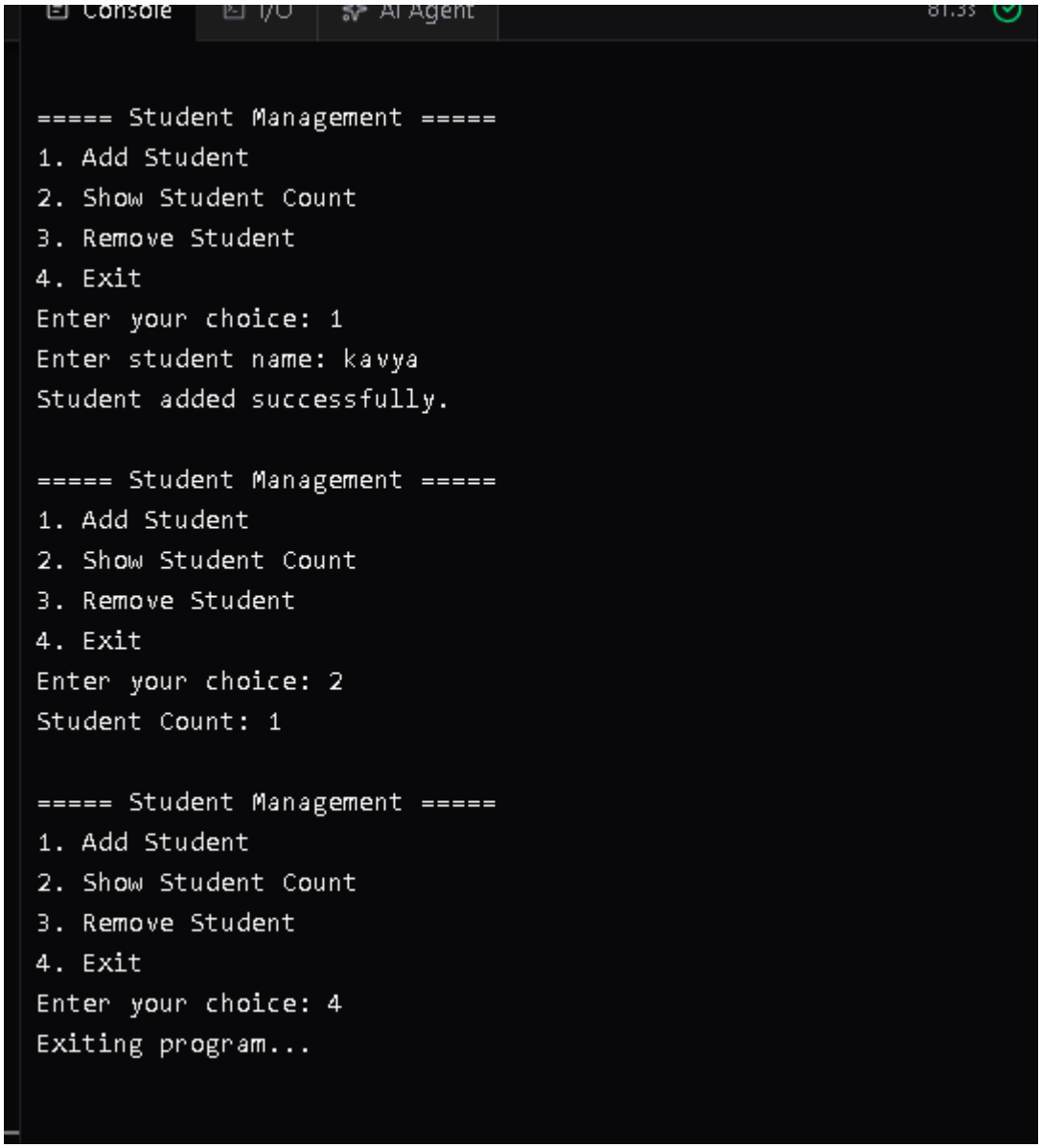
                    try {
                        service.removeStudent(index);
                        System.out.println("Student removed successfully.");
                    } catch (IllegalArgumentException e) {
                        System.out.println(e.getMessage());
                    }
                    break;

                case 4:
                    System.out.println("Exiting program...");
                    sc.close();
                    return;

                default:
                    System.out.println("Invalid choice.");
            }
```

>_ Terminal

Output :



Learning Outcome – Problem 1

- Understand the **layered structure** of a Java application.
- Learn to create a **Model class** and a **Service class**.
- Use **ArrayList** to store student objects.
- Perform operations like **add, remove, and count**.
- Learn to write **JUnit test cases** for checking program functionality.
- Understand how testing helps find **invalid actions and errors**.

Aim – 2 : *For Advanced Learners – home*

Organise that app into packages, build a runnable JAR, and add a test that saves and reloads its data unchanged.

Code :

Main.java

```java
import java.io.*;
import java.util.ArrayList;

// Model
class Student implements Serializable {

    private String name;

    public Student(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}

// Service
class StudentService {

    private ArrayList<Student> students = new ArrayList<>();

    // Add student
    public void addStudent(Student student) {
        students.add(student);
    }

    // Get count
    public int getCount() {
        return students.size();
```

ady >_ Terminal

CEUE20325IT103

Main.java

```
class StudentService {
    // Save data
    public void saveData(String fileName) throws IOException {

        ObjectOutputStream out =
                new ObjectOutputStream(
                        new FileOutputStream(fileName));

        out.writeObject(students);
        out.close();

        System.out.println("Data saved successfully.");
    }

    // Load data
    @SuppressWarnings("unchecked")
    public void loadData(String fileName)
            throws IOException, ClassNotFoundException {

        ObjectInputStream in =
                new ObjectInputStream(
                        new FileInputStream(fileName));

        students = (ArrayList<Student>) in.readObject();
        in.close();

        System.out.println("Data loaded successfully.");
    }

    public void displayStudents() {
```

Terminal

Main.java

```
class StudentService {
    }

    public void displayStudents() {

        for (Student student : students) {
            System.out.println(student.getName());
        }
    }
}

// Main
public class Main {

    public static void main(String[] args) {

        try {

            // Create service
            StudentService service = new StudentService();

            // Add students
            service.addStudent(new Student("Kavya"));
            service.addStudent(new Student("Rahul"));

            System.out.println("Original Data:");
            service.displayStudents();

            System.out.println("Student Count: "
                    + service.getCount());
```

Ready Terminal

Main.java

```
public class Main {
    public static void main(String[] args) {
        try {
            System.out.println("Original Data:");
            service.displayStudents();

            System.out.println("Student Count: "
                    + service.getCount());

            // Save data
            service.saveData("students.dat");

            // Create new service
            StudentService loadedService =
                    new StudentService();

            // Load saved data
            loadedService.loadData("students.dat");

            System.out.println("\nReloaded Data:");
            loadedService.displayStudents();

            System.out.println("Student Count After Reload: "
                    + loadedService.getCount());

        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

ady >_ Terminal

Output :

```
Original Data:
Kavya
Rahul
Student Count: 2
Data saved successfully.
Data loaded successfully.

Reloaded Data:
Kavya
Rahul
Student Count After Reload: 2
```

Learning Outcome – Problem 2

- Understand how to **organize Java code into packages**.
- Learn to create a **runnable JAR file**.
- Learn to **save and reload data** using serialization.
- Verify that data remains **unchanged after reloading**.
- Understand how **JUnit testing** can verify data persistence.